

Core Java — Coding Practice Set

Lectures 1–5 | Coder Army 'Java Full Course 2026'
Prepared for Pandra — Path to Full Stack Java Developer

Scope of this set: Only concepts from Lecture 1 (Intro to Java, JVM/bytecode), Lecture 2 (First program, JDK/JRE/JVM, JSE/JEE/JME), Lecture 3 (Variables, Data Types, Identifiers, Literals, Keywords), Lecture 4 (Negative & Floating-Point Storage), and Lecture 5 (Type Conversion & Casting). **No loops, conditionals, or operators are used** anywhere in this set — those start from Lecture 6 onward and will appear in the next batch.

How to use this set: Solve each question in a .java file, compile with javac, run with java. Try to answer/predict before running the code. Send me your solutions/screenshots and I'll review them, point out mistakes, and rate your code quality.

LEVEL 1 — EASY (Q1–Q20)

Goal: comfort with declaring variables, correct literals, and printing output.

Q1. Declare an **int** variable named *age*, assign your age to it, and print it using `System.out.println`.

Hint: Recall the syntax: `dataType variableName = value;` — think about what a valid int value looks like.

Q2. Declare a **char** variable named *grade* and assign a single letter to it, then print it.

Hint: Char literals use single quotes, not double quotes.

Q3. Declare a **double** variable named *price* and assign a decimal value, then print it.

Hint: Decimal literals default to double in Java — no suffix needed.

Q4. Declare a **boolean** variable named *isJavaFun*, set it to true, and print it.

Hint: Boolean only has two legal literal values — think about what they're called, not 1/0.

Q5. Declare a **String** variable named *city* with your city's name, and print it.

Hint: String literals use double quotes; remember String is not a primitive type.

Q6. Declare a **byte** variable and assign it the maximum value a byte can legally hold, then print it.

Hint: Think back to Lecture 4 — how many bits does a byte have, and what's the range formula for a signed type?

Q7. Declare a **long** variable named *population* and assign a large value that needs a special suffix, then print it.

Hint: Without the right suffix, a large whole number literal is treated as int by default and may not compile.

Q8. Declare a **float** variable named *temperature* with a decimal value, then print it.

Hint: Decimal literals are double by default — what do you need to add so it's accepted as float?

Q9. Print the exact text *Hello, Java!* to the console.

Hint: Just `System.out.println` with the right String literal.

Q10. In a single line, declare an int, a char, and a boolean variable (comma-separated), then print all three.

Hint: Java allows multiple variables of the same type to be declared in one statement, separated by commas — but here you have three different types, so think about whether one line is even legal, and if not, how you'd cleanly declare three in as few lines as possible.

Q11. Declare a **final** variable named *PI* of type double with value 3.14159, then print it.

Hint: The 'final' keyword marks a variable as a constant — its value cannot be changed after assignment.

Q12. Declare an `int` variable *without* assigning a value on the same line, then assign a value on the next line, then print it.

Hint: Java allows declaration and initialization to be separate statements, as long as a value exists before you use the variable.

Q13. Declare a `short` variable and assign it the smallest (most negative) legal value it can hold, then print it.

Hint: Short is a signed 16-bit type — recall how the range is split between negative and positive using two's complement.

Q14. Declare a `char` variable using a Unicode escape sequence (e.g. representing the letter A), then print it.

Hint: Unicode escapes look like `\u` followed by 4 hex digits, written inside single quotes like any char literal.

Q15. Declare an `int` variable using an `octal` literal (a number starting with 0), then print its decimal value.

Hint: A leading 0 before digits tells the Java compiler to interpret the number in base-8.

Q16. Declare an `int` variable using a `hexadecimal` literal (starting with 0x), then print its decimal value.

Hint: 0x tells the compiler the digits that follow are base-16.

Q17. Declare an `int` variable using a `binary` literal (starting with 0b), then print its decimal value.

Hint: 0b is the prefix Java uses for binary literals — introduced as an alternative to hex/octal.

Q18. Declare a large int literal using underscores for readability (e.g. a value in the ten-millions), then print it.

Hint: Underscores can be placed between digits purely for human readability; they don't change the value.

Q19. Write a short program that stores the keywords 'public', 'static', 'void', 'main' as a single comment above your main method, explaining in your own words what each keyword contributes to your program's entry point.

Hint: Think back to Lecture 1/2 — this isn't about writing new logic, just recalling what each keyword in the method signature means.

Q20. Declare an `int` variable and try giving it a value outside the `int` range using a plain number literal (no suffix) — note the compiler error you get, then fix it properly with the correct type and/or suffix.

Hint: This is about diagnosing why the compiler rejects a plain literal that's too large for `int`, and choosing the right fix — a different data type or a suffix.

LEVEL 2 — MEDIUM (Q21–Q40)

Goal: type casting, widening/narrowing, predicting output based on Lecture 4 & 5 concepts.

Q21. Store an int value larger than 127 into a byte variable using explicit casting. Print the byte's value and explain in a comment why it doesn't match the original number.

Hint: Byte can only hold 8 bits — think about what happens to the extra higher-order bits when they don't fit, tying back to two's complement from Lecture 4.

Q22. Convert a double value like 9.75 into an int using explicit casting. Print the result and explain what happened to the .75 part.

Hint: Narrowing from a floating type to an integer type doesn't round — think about what simple operation it performs instead.

Q23. Assign a char variable to an int variable without any explicit cast, then print the int value.

Hint: Char to int is a widening conversion — recall which conversions Java allows implicitly.

Q24. Cast an int value to a char explicitly and print the resulting character.

Hint: Every char in Java corresponds to a numeric code point — casting an int to char looks that code point up.

Q25. Try assigning a small int literal (e.g. 100) to a byte variable without a cast, then try a large int literal (e.g. 500) the same way. Note which one compiles and which doesn't.

Hint: Java allows small literal values to auto-narrow into byte/short only if the compiler can verify they fit — think about why 500 breaks that rule.

Q26. Declare a float variable and assign it a decimal literal without any suffix. Note the compiler error, then fix it.

Hint: Recall the default type of a plain decimal literal, and what suffix forces it to be treated as float instead.

Q27. Store Integer's maximum legal int value in a variable, then explicitly cast it to short and print the result.

Hint: Short only has 16 bits — think about what portion of the original 32 bits actually survives the cast.

Q28. Declare a byte variable, assign it a small value, then assign it to an int variable without a cast. Print both.

Hint: This is a widening conversion, which Java always allows implicitly — no cast needed.

Q29. Declare a long variable with a large value, then explicitly cast it down to int and print both values.

Hint: Narrowing a 64-bit value into 32 bits — think about which bits get kept and which get dropped.

Q30. Declare a variable using `var` with an int literal, and in a comment state what type Java actually infers for it.

Hint: `var` doesn't mean 'no type' — it means the compiler figures out the type for you at compile time from the assigned value.

Q31. Cast the value 130 to a byte using `(byte)130` and print it. In a comment, explain the printed value using two's complement wraparound from Lecture 4.

Hint: 130 doesn't fit in a signed byte's positive range — trace what bit pattern 130 has in 8 bits and how that pattern is interpreted as a signed number.

Q32. Cast the value -200 to a byte and print it. Explain the result in a comment.

Hint: Same wraparound idea as above, but starting from a negative number this time — think about how negative numbers are stored in binary.

Q33. Cast a float value like 9.99f to a long and print the result. Explain in a comment what happened to the decimal part.

Hint: Casting float/double to an integer type always truncates toward zero — it never rounds.

Q34. Cast a double literal 9.99 to an int directly (no intermediate variable) inside the println statement, and print the result.

Hint: You can cast directly inside an expression — the cast applies to the literal value before it's passed to println.

Q35. Declare an int variable, implicitly widen it into a double variable, then explicitly narrow that double back into a new int variable. Print all three and comment on whether the final int matches the original.

Hint: Going int → double is always safe, but double → int truncates any decimal part introduced along the way — think about whether an int-turned-double ever gains a decimal part.

Q36. Declare a char variable with value corresponding to 65, print it directly as a character, then cast it to int and print that.

Hint: The same variable's bits don't change — only how println interprets and displays them changes based on the type of the expression you give it.

Q37. Take an int with value -1, cast it to byte, then cast that byte back into a new int variable. Predict and print the final printed value.

Hint: This tests 'sign extension' — when a negative byte is widened back to int, Java preserves the sign, which may surprise you if you expected it to 'reset'.

Q38. Declare an int using a binary literal representing the value 5, cast it to byte, then cast that byte to char, and print the resulting character.

Hint: Chain the casts one at a time and think about what small integer values correspond to as ASCII characters.

Q39. Store the same numeric value (e.g. 1234567890) in both an int and a long variable, then cast the long down to int and compare. Print all values.

Hint: Since the value already fits in int's range, think about whether the round-trip conversion should lose anything here — contrast this with the earlier questions where it did.

Q40. Declare a double value like 123456.789123, then cast it to float and print both. Comment on which one prints with more precision.

Hint: Float has fewer bits reserved for precision than double (see Lecture 4's floating-point storage) — that difference should show up in longer decimal values.

LEVEL 3 — HARD (Q41–Q52)

Goal: combine Lecture 3 + 4 + 5 concepts, edge cases, and 'why' reasoning — strong interview-prep territory.

Q41. Take a long literal larger than `Integer.MAX_VALUE`, and cast it step-by-step: `long` → `int` → `short` → `byte`, printing the value at every stage. Comment on what happens at each narrowing step.

Hint: Each step chops off higher-order bits from the previous binary representation — trace the bit pattern truncation stage by stage rather than just guessing the final number.

Q42. Cast a very large double (something like `1e20`, way beyond `long`'s range) to a `long`, print it, and explain the result in a comment.

Hint: Unlike integer-to-integer narrowing, casting an out-of-range floating value to a whole number type in Java doesn't wrap around — it 'saturates' at a boundary value. Think about which boundary.

Q43. Declare `char c` with value corresponding to `-1` cast into `char`, print it as an `int`. Explain why the result differs from what `-1` would look like as a `byte`.

Hint: `Char` is Java's only unsigned primitive type (16-bit, 0 to 65535) — think about how the same bit pattern gets interpreted differently for a signed vs an unsigned type.

Q44. Declare a `final` variable, then attempt to reassign it in a separate statement further down (leave this second assignment as a commented-out line since it won't compile). Explain in a comment why Java blocks it.

Hint: '`final`' isn't just a style convention — think about what guarantee the compiler is trying to give any code that reads that variable later.

Q45. Declare a `char` variable holding the letter 'Z', print its underlying `int` value, then declare a second `char` variable directly assigned the `int` value that corresponds to the next letter of the alphabet ('[' in ASCII terms doesn't count — think carefully about what number that actually is), and print that character.

Hint: You don't have arithmetic operators yet, so you'll need to reason out the correct numeric code by hand rather than compute it — this is really testing whether you understand the ASCII/Unicode ordering of letters.

Q46. Write a short program (as comments only, no actual illegal code) explaining why Java does NOT allow assigning a `boolean` directly to an `int`, contrasting this with how C/C++ treats `true/false` as `1/0` (tie back to Lecture 1).

Hint: This is a conceptual/theory answer captured in comments — focus on Java's strict type system as a deliberate design decision, not a limitation.

Q47. Declare the same fractional value (e.g. `1.1`) once as a `float` and once as a `double`, print both with many decimal places, and explain any difference you observe.

Hint: Because `1.1` can't be represented exactly in binary floating point, the rounding error differs based on how many bits of precision the type reserves — `float` has noticeably fewer than `double`.

Q48. Declare an `int` variable holding a value, and a `String` variable holding a piece of text. In comments, describe how each is likely stored in memory (stack vs heap) based on the difference between primitive and reference types.

Hint: This is conceptual — primitives store their actual value directly, while non-primitives like `String` store a reference to an object elsewhere in memory.

Q49. Assign an oversized `int` literal to a `byte` using an explicit cast where the value, in binary, happens to have its highest bit set (making the result negative). Choose your own number greater than 127 and less than 256, and predict the sign of the output before running it.

Hint: The 8th bit (the sign bit) of a `byte` determines positive vs negative — pick a value where that particular bit ends up set to 1 once truncated to 8 bits.

Q50. Explain, using only comments (no loops/conditionals yet), why widening conversions (like int to long) never need an explicit cast, but narrowing conversions (like long to int) always do — relate this to potential data loss.

Hint: The compiler enforces a cast for narrowing conversions specifically because they can silently lose information — an explicit cast is your way of telling the compiler 'I understand the risk.'

Q51. Declare four variables of type byte, short, int, and long, each holding the same small value (e.g. 10), then implicitly widen each one step-by-step into the next larger type, printing every intermediate variable.

Hint: Chain the assignments one type at a time (byte→short→int→long) to clearly see that no casts are required anywhere in this direction.

Q52. A classmate writes: *byte b = 10; byte c = 20; byte d = b + c;* and says it won't compile even though operators aren't part of this batch. In comments only, predict whether this compiles or not, and explain why, based on what you know so far about byte's range and Java's default numeric promotion rules.

Hint: This is a forward-looking theory question — you haven't formally learned + yet, but you already know enough about byte's range and int being Java's 'default' numeric type to reason about the answer.

52 questions total — 20 Easy, 20 Medium, 12 Hard. Solve at your own pace across the next few days alongside your daily quiz. Send me completed code (or a summary of your answers) whenever you're ready to review.